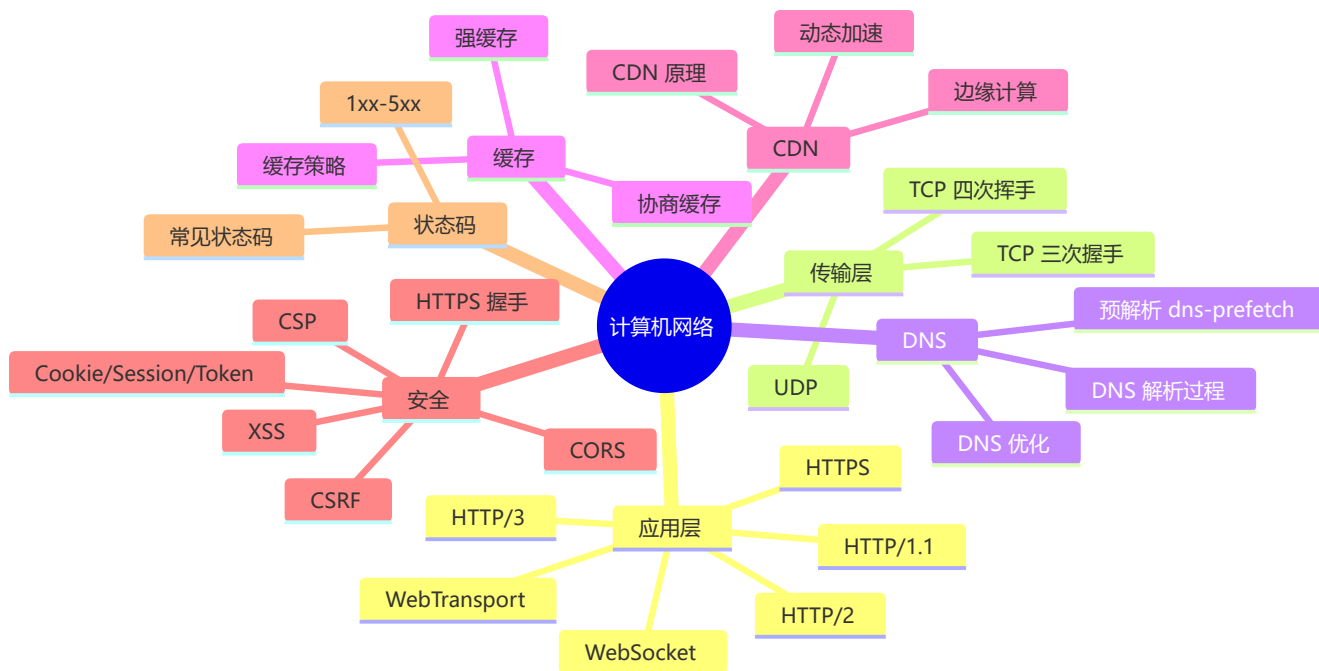


🌐 计算机网络：前端面试必备

🎯 面试星级：★★★★☆ | 建议用时：2 天

协议栈 + 安全 + CDN + 缓存 + CORS — 前端日常碰到的网络知识全在这儿

📌 知识脑图



📄 目录

- [一、HTTP 发展史](#)
- [二、HTTPS 与 TLS](#)
- [三、TCP 与 UDP](#)
- [四、DNS 解析](#)
- [五、浏览器缓存机制](#)
- [六、CDN 原理与实践](#)
- [七、CORS 跨域](#)
- [八、网络安全（前端视角）](#)
- [九、Cookie / Session / Token](#)
- [十、HTTP 状态码](#)
- [十一、WebSocket 与 SSE](#)
- [十二、HTTP/3 与 WebTransport](#)
- [十三、面试高频问答](#)

一、HTTP 发展史

1.1 HTTP/0.9 → HTTP/1.1

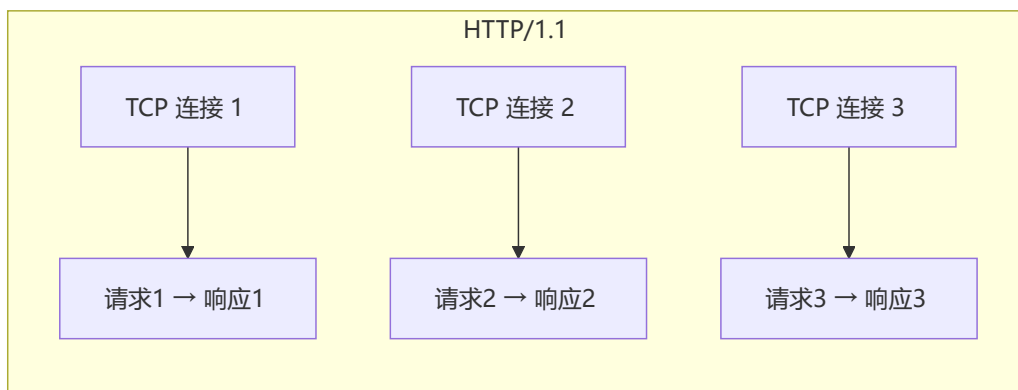
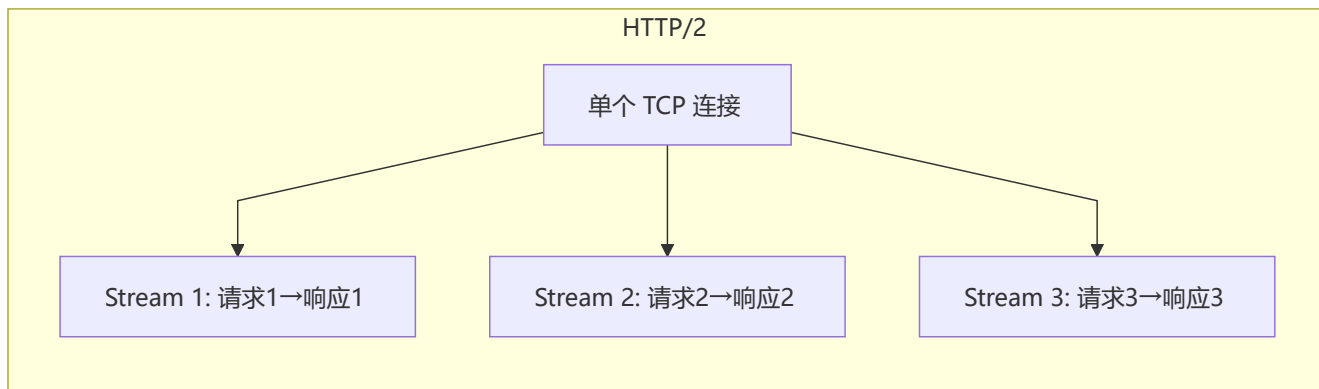
版本	时间	关键特性
HTTP/0.9	1991	仅 GET 请求，纯文本响应，无头部
HTTP/1.0	1996	增加 Header、状态码、Content-Type，每个请求新建 TCP 连接
HTTP/1.1	1997	持久连接（Keep-Alive）、管线化（Pipelining）、Host 头、分块传输

HTTP/1.1 核心问题：

- **队头阻塞**：管线化虽支持并发请求，但响应必须按顺序返回
- **头部冗余**：每次请求携带完整头部，Cookie/User-Agent 等重复发送
- **不支持优先级**：请求无法标记优先级

1.2 HTTP/2

特性	说明
二进制分帧	取代文本传输，解析效率更高
多路复用	一个 TCP 连接并行多个 Stream
头部压缩	HPACK 算法，减少冗余头部
服务端推送	Server Push（已废弃，Chrome 106 移除）



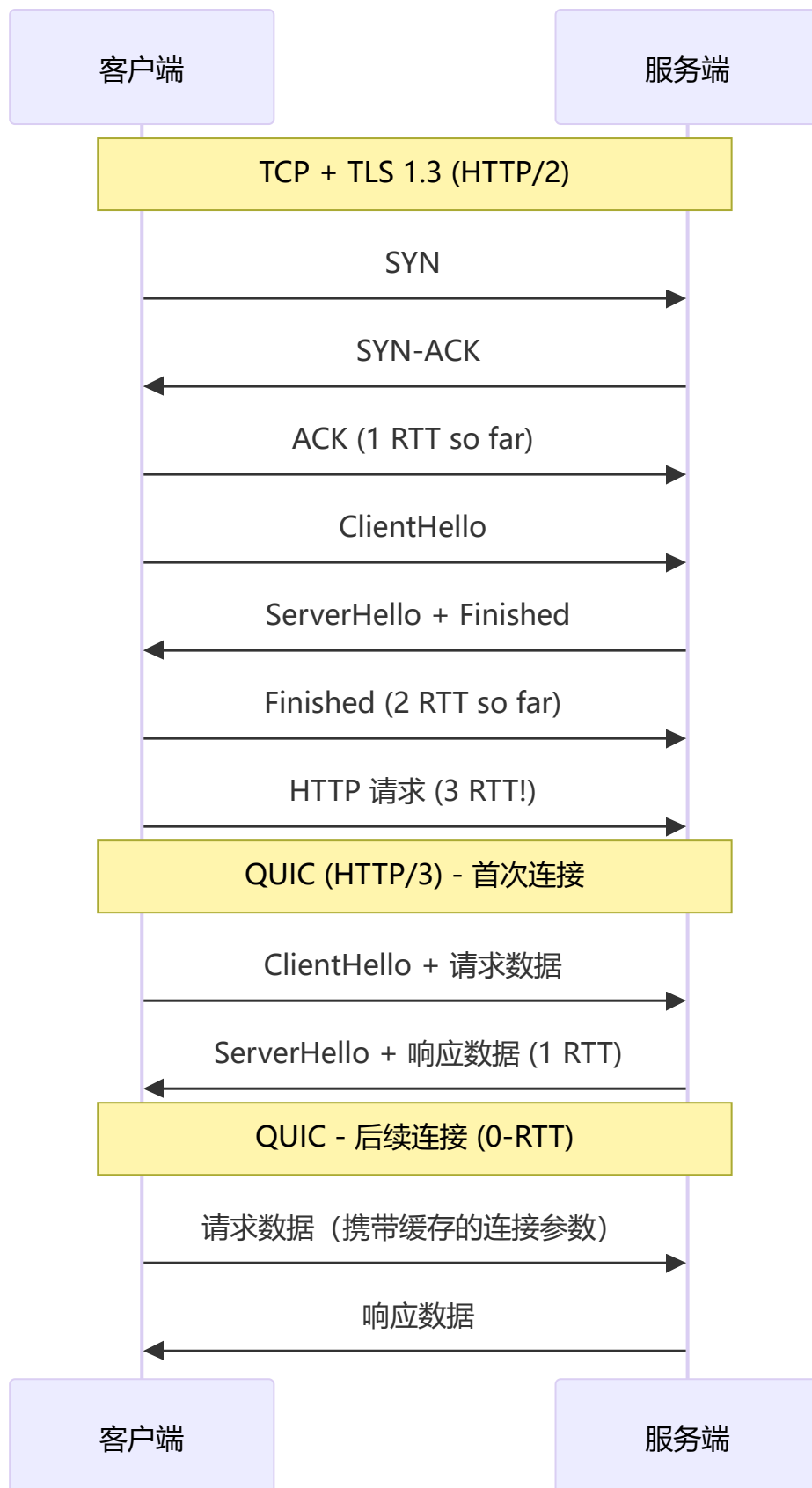
HTTP/2 的问题:

- **TCP 层面的队头阻塞:** 虽然 Stream 层解决了, 但 TCP 层丢包会导致所有 Stream 阻塞
- **连接数限制:** 单个域名并发连接数受限
- **握手延迟:** TCP + TLS 需要 2-3 个 RTT

1.3 HTTP/3

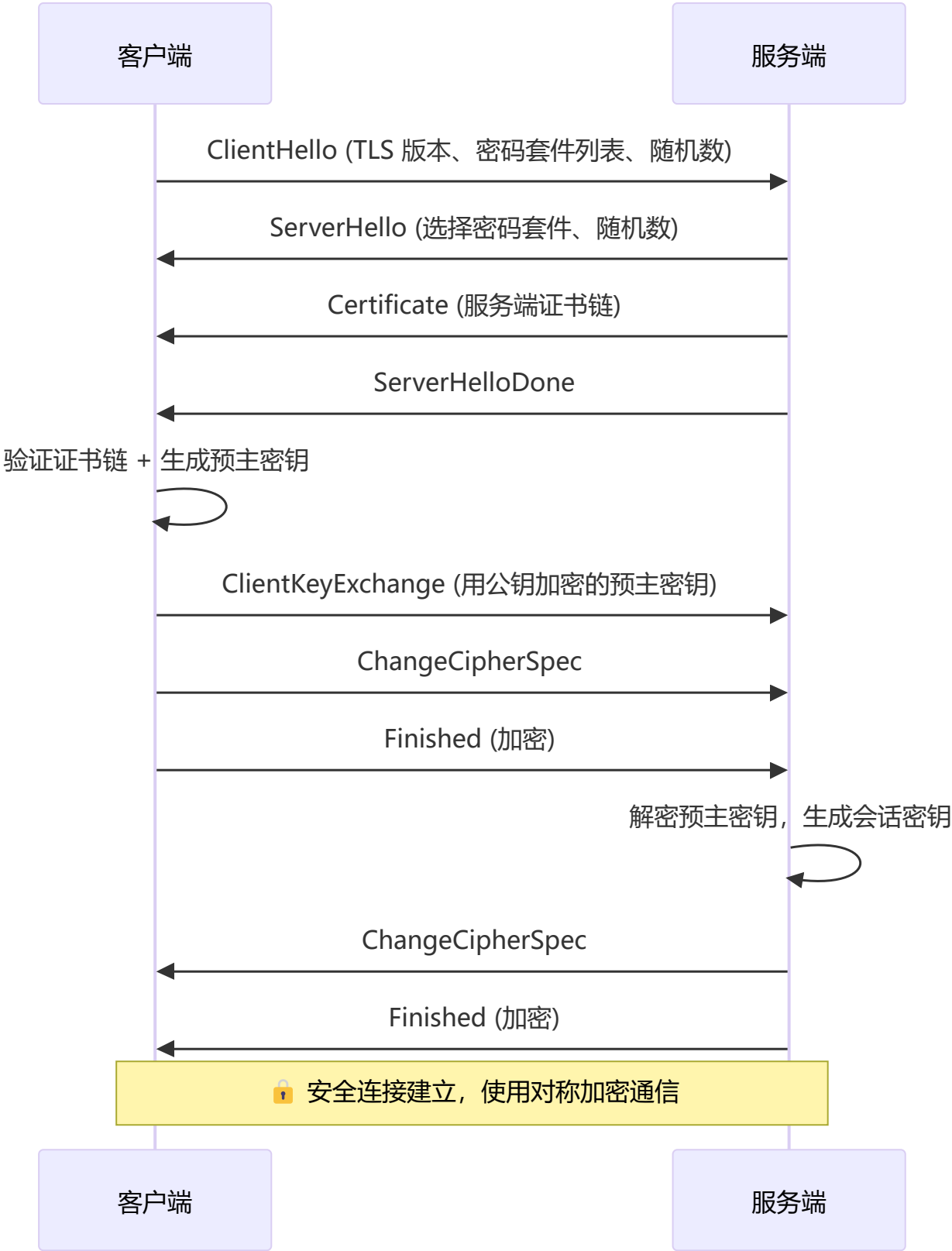
基于 **QUIC** (Quick UDP Internet Connections) 协议:

特性	说明
基于 UDP	避免 TCP 队头阻塞
0-RTT 连接	首次 1-RTT, 后续 0-RTT
内置 TLS 1.3	加密默认、不可降级
连接迁移	切换网络时连接不中断
独立流	一个流丢包不影响其他流



二、HTTPS 与 TLS

2.1 TLS 握手过程



2.2 为什么用混合加密？

加密方式	优点	缺点
对称加密	速度快、适合大数据	密钥分发不安全
非对称加密	公钥可公开、密钥分发安全	速度慢、不适合大数据

HTTPS 方案：非对称加密传输对称密钥 → 对称加密通信数据

2.3 证书验证链

浏览器内置根证书 → 根 CA 签发 → 中间 CA → 服务器证书

↓

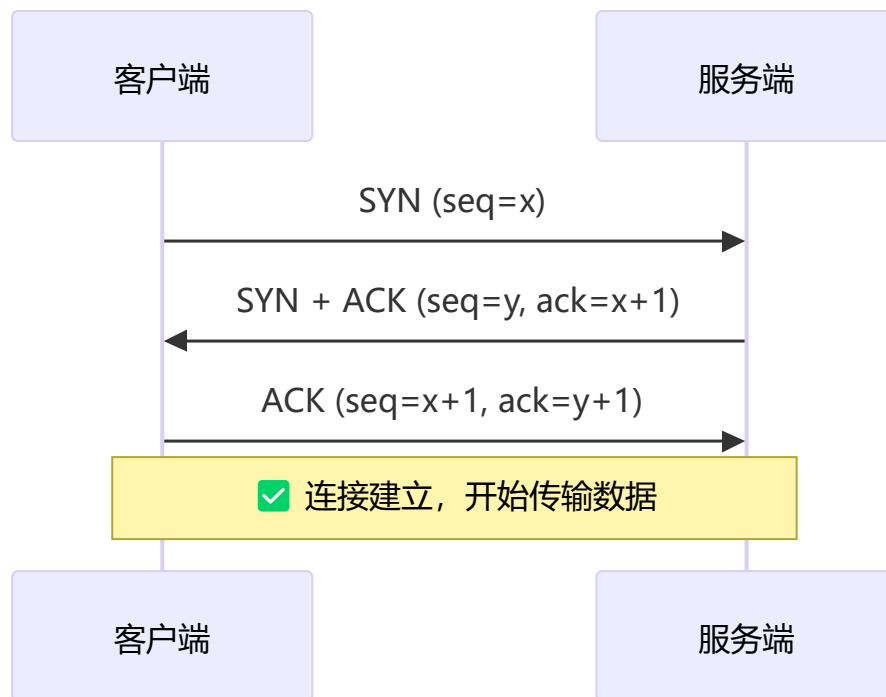
浏览器验证签名、有效期、域名、吊销状态（OCSP）

2.4 面试追问

- **Q：**HTTPS 一定安全吗？
- **A：**HTTPS 保证传输加密，但不保证服务器可信。中间人可以通过伪造证书攻击（如代理软件安装根证书）。
- **Q：**如何验证证书是否被吊销？
- **A：**OCSP（在线证书状态协议）或 CRL（证书吊销列表）。
- **Q：**HTTPS 对性能有多大影响？
- **A：**现代环境下 HTTPS 性能开销 <1%：
 - **加密计算：**AES-GCM 有 CPU 硬件指令集（AES-NI）加速，开销可忽略
 - **握手开销**（主要瓶颈）：TLS 1.2 需 2 RTT，TLS 1.3 优化为 1 RTT，支持 0-RTT 会话恢复
 - **优化手段：**Session Resumption（Session ID / Ticket）、TLS False Start、HTTP/2 多路复用、HTTP/3 基于 QUIC 实现 0-RTT 建连
 - **实际影响：**首次访问增加 10-50ms（取决于 RTT），后续访问通过会话复用几乎无感知

三、TCP 与 UDP

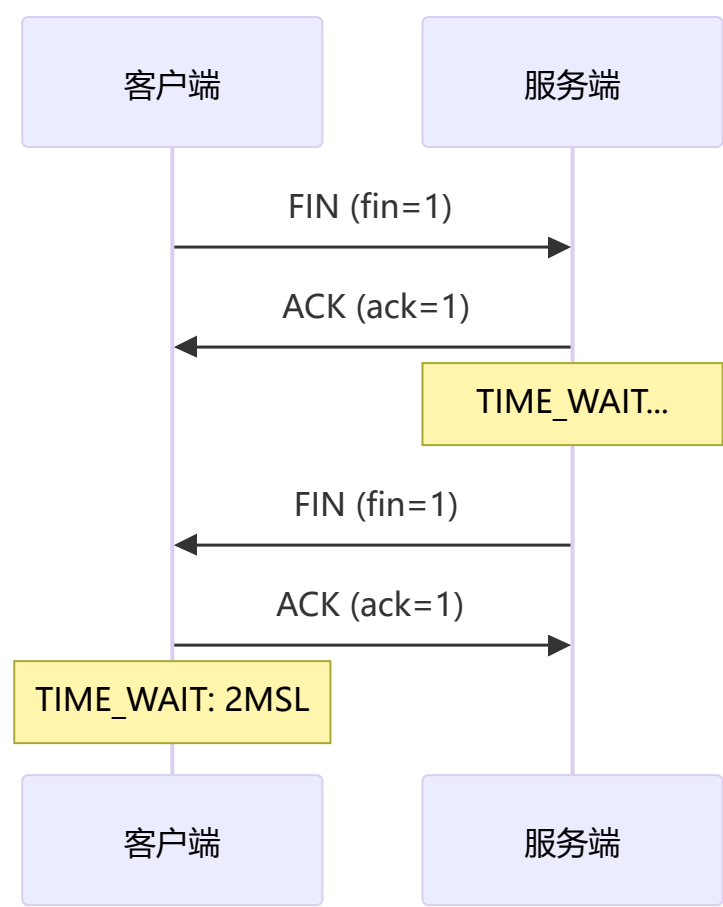
3.1 TCP 三次握手



为什么是三次而不是两次？

- 防止已失效的连接请求突然传到服务端
- 两次握手 → 服务端无法确认客户端是否收到自己的 SYN+ACK
- 三次握手双方都能确认对方的接收能力正常

3.2 TCP 四次挥手



3.3 TCP 拥塞控制

- 慢启动:** 初始发送窗口为 1，每收到一个 ACK 翻倍，直到阈值
- 拥塞避免:** 超过阈值后线性增长
- 快速重传:** 收到 3 个重复 ACK 就重传丢失包
- 快速恢复:** 重传后降低阈值，继续拥塞避免

3.4 TCP vs UDP

对比项	TCP	UDP
连接性	面向连接	无连接
可靠性	可靠（重传、确认）	不可靠（可能丢包）
顺序	保证顺序	不保证顺序
速度	较慢	快
头部大小	20-60 字节	8 字节
应用	HTTP、FTP、SMTP	DNS、VoIP、视频流、QUIC

四、DNS 解析

4.1 解析流程

用户在浏览器输入 `www.example.com`

↓

1. 浏览器缓存 → 有则返回

2. 操作系统缓存 → `hosts` 文件

3. 本地 DNS 服务器（如 `114.114.114.114`）

4. 根域名服务器 → 查 `.com` 的 NS

5. `.com` 顶级域名服务器 → 查 `example.com` 的 NS

6. `example.com` 权威 DNS → 返回 IP

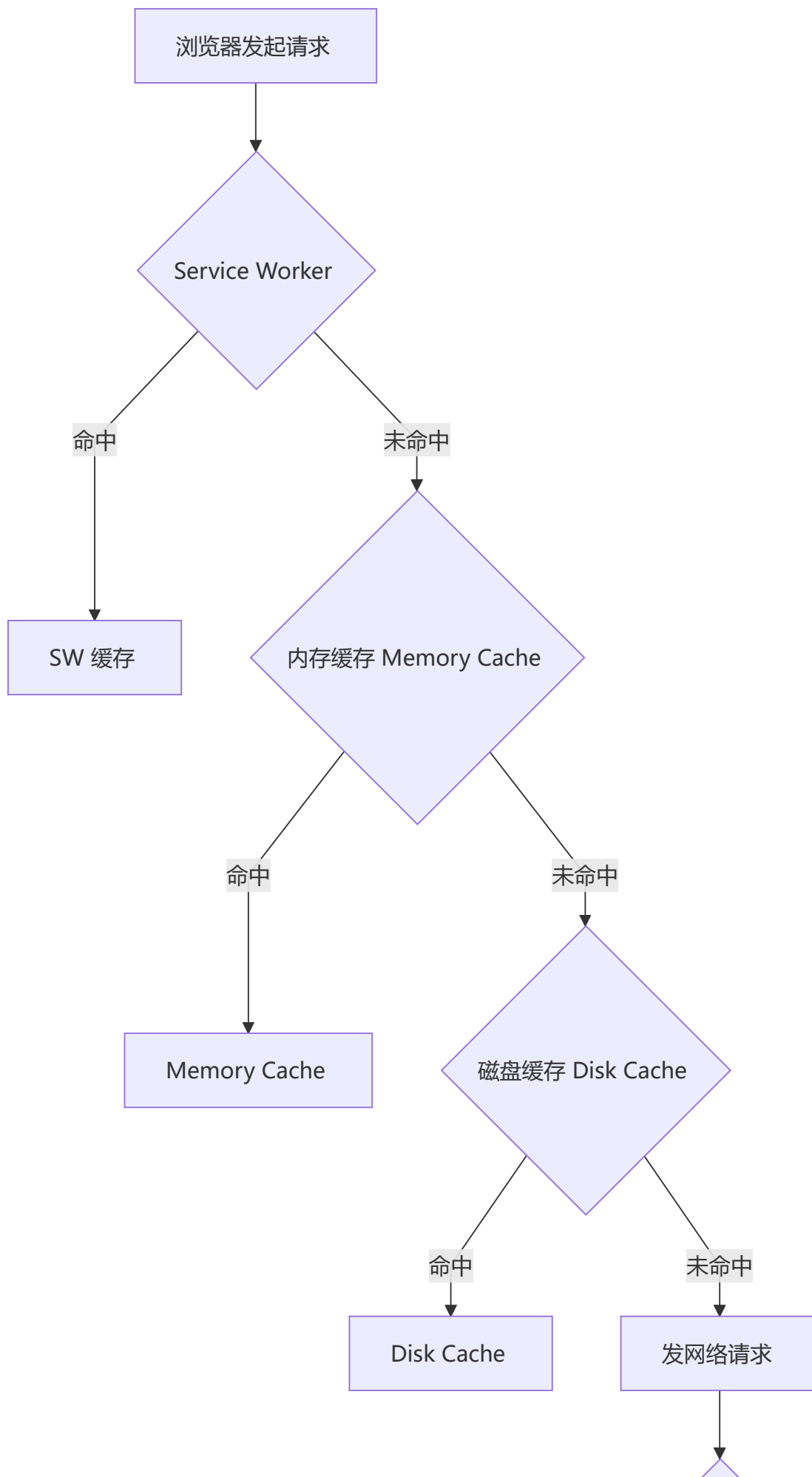
4.2 DNS 优化

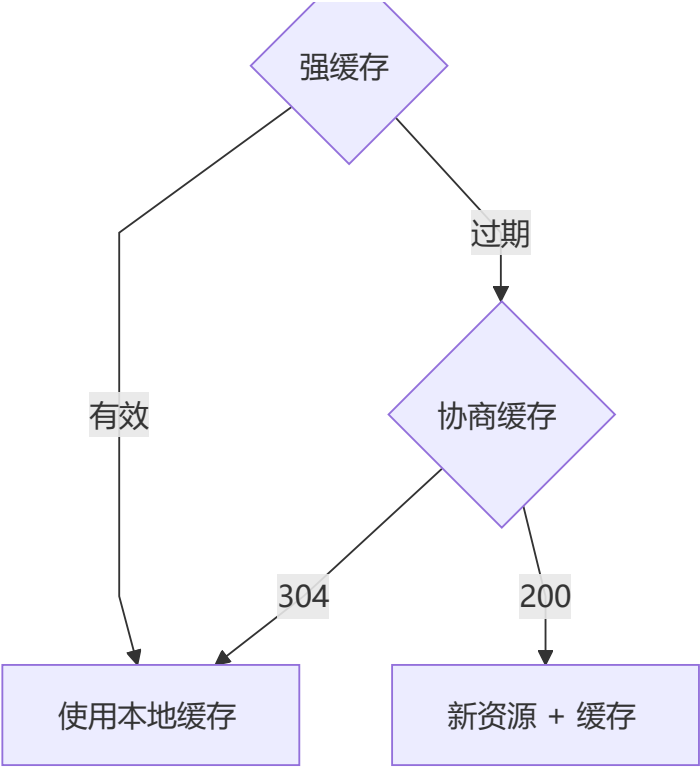
优化手段	说明
<code>dns-prefetch</code>	提前解析页面中将要使用的域名
<code>preconnect</code>	不仅解析 DNS，还完成 TCP/TLS 握手
CDN 智能调度	GEO DNS 就近返回节点 IP

```
<!-- DNS 预解析 -->  
<link rel="dns-prefetch" href="//cdn.example.com">  
<!-- 预连接（更激进） -->  
<link rel="preconnect" href="//api.example.com">
```

五、浏览器缓存机制

5.1 缓存位置





5.2 强缓存

头部	说明	优先级
Cache-Control	max-age=3600 相对时间	高
Expires	Expires: Wed, 22 Oct 2025 07:28:00 GMT 绝对时间	低 (HTTP/1.0)

Cache-Control 常用指令:

max-age=seconds

→ 缓存有效时间

public

→ 可被中间缓存缓存

private

→ 仅浏览器可缓存

no-cache

→ 使用前需验证 (协商缓存)

no-store

→ 禁止任何缓存

must-revalidate

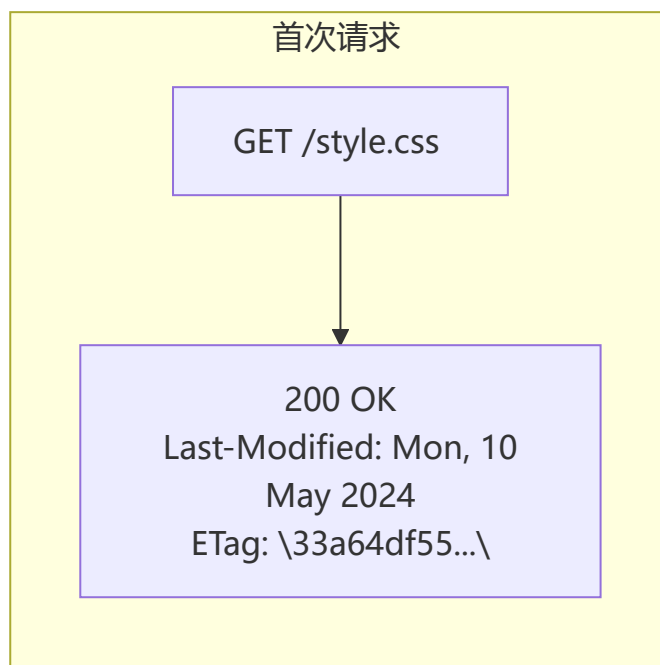
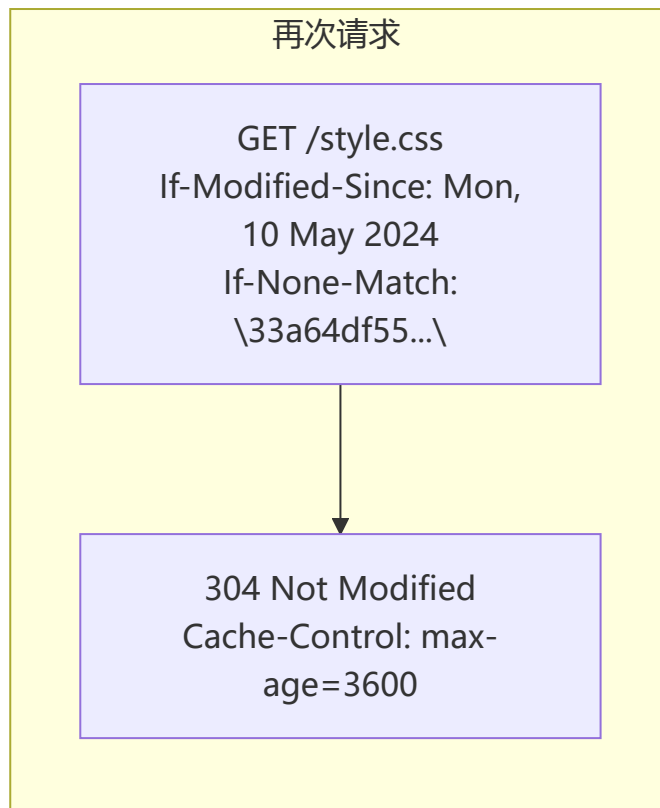
→ 过期后必须验证

immutable

→ 资源永不更新 (配合版本号)

5.3 协商缓存

头部对	方向	原理
Last-Modified / If-Modified-Since	响应/请求	基于文件修改时间 (秒级)
ETag / If-None-Match	响应/请求	基于文件内容哈希 (精确)



5.4 最佳缓存策略

资源类型	策略	原因
HTML	<code>no-cache</code>	内容频繁更新
JS/CSS	<code>max-age=31536000 + 文件名哈希</code>	版本号变化强制更新

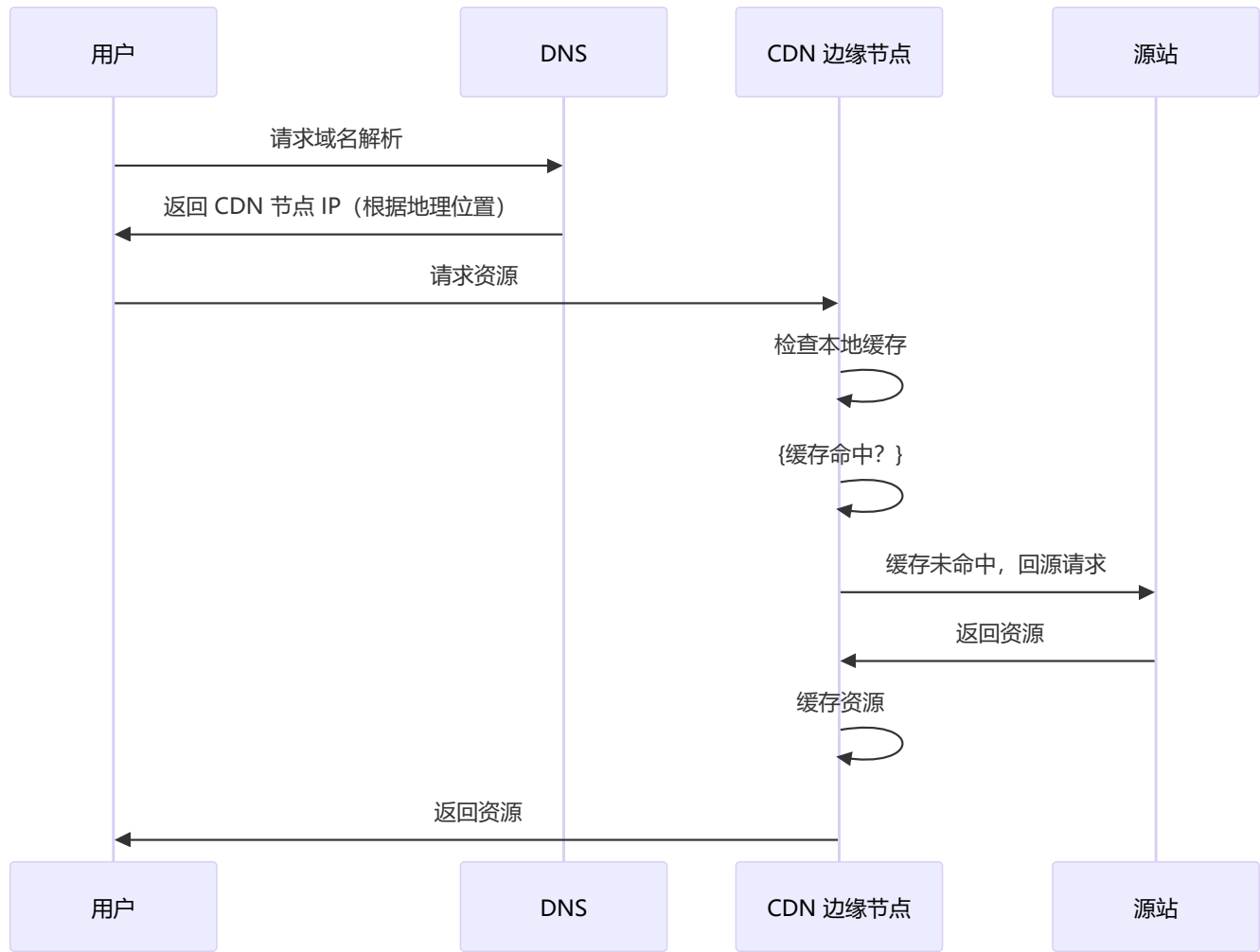
资源类型	策略	原因
图片/字体	<code>max-age=3600 + ETag</code>	可长期缓存
API 响应	<code>no-cache</code> 或短期 <code>max-age</code>	数据需要及时更新

```
# Nginx 配置示例
location /static/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
}

location /api/ {
    add_header Cache-Control "no-cache";
}
```

六、CDN 原理与实践

6.1 CDN 工作流程



6.2 CDN 优化策略

策略	说明
预热	提前将热门资源推送到 CDN 节点
动态加速	DCDN，优化动态请求的传输路径
边缘计算	Cloudflare Workers、Edge Functions 在 CDN 节点执行代码
多 CDN	多个 CDN 提供商冗余，故障切换

6.3 CDN 适合缓存的内容

内容	缓存策略
静态资源 (JS/CSS/图片)	长期缓存 + 版本号
字体文件	长期缓存 (字体很少变)
视频流	分片缓存
API 响应	短期缓存或不缓存
HTML 页面	协商缓存

七、CORS 跨域

7.1 同源策略

同源 = 协议 + 域名 + 端口 三者完全相同

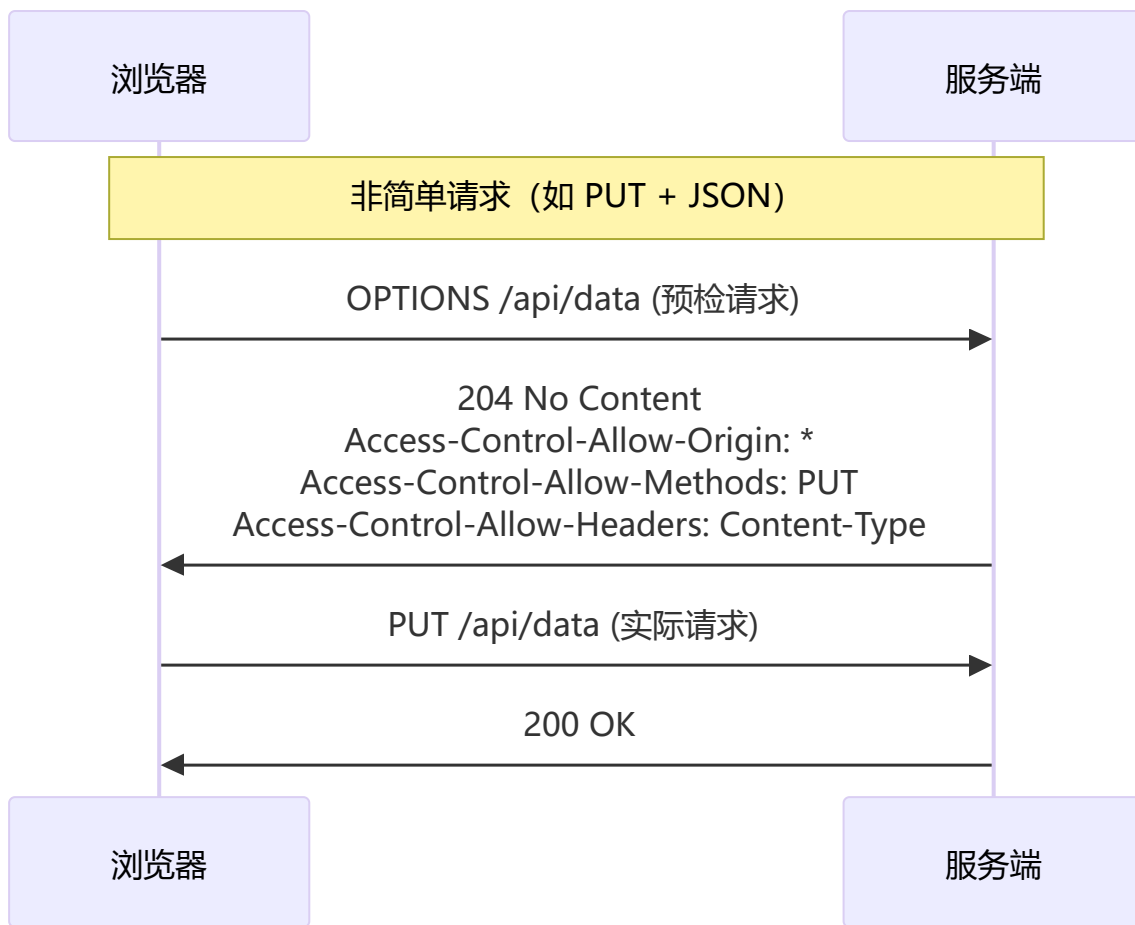
页面 URL	请求 URL	是否同源
https://a.com/page	https://a.com/api	✓
https://a.com/page	http://a.com/api	✗ (协议不同)
https://a.com/page	https://b.com/api	✗ (域名不同)
https://a.com/page	https://a.com:8080/api	✗ (端口不同)

7.2 简单请求与非简单请求

简单请求条件 (全部满足)：

- 1. Method: GET / HEAD / POST
- 2. Content-Type: text/plain / multipart/form-data / application/x-www-form-urlencoded
- 3. 无自定义头部

非简单请求：浏览器先发 OPTIONS 预检请求



7.3 跨域相关头部

响应头	说明	示例
<code>Access-Control-Allow-Origin</code>	允许的源	<code>*</code> 或 <code>https://a.com</code>
<code>Access-Control-Allow-Methods</code>	允许的方法	<code>GET, POST, PUT</code>
<code>Access-Control-Allow-Headers</code>	允许的请求头	<code>Content-Type, Authorization</code>
<code>Access-Control-Allow-Credentials</code>	是否允许带 Cookie	<code>true</code> (此时 Origin 不能为 <code>*</code>)
<code>Access-Control-Max-Age</code>	预检结果缓存时间	<code>86400</code> (24 小时)
<code>Access-Control-Expose-Headers</code>	允许 JS 读取的响应头	<code>X-Total-Count</code>

7.4 常见跨域解决方案

方案	适用场景	优缺点
CORS	后端可控	最标准, 支持所有 HTTP 方法
JSONP	仅 GET、后端不可加头	只能 GET, 有安全风险
Proxy 代理	开发环境 (Webpack/Vite Proxy)	配置简单, 生产环境需 Nginx

方案	适用场景	优缺点
Nginx 反向代理	生产环境同源策略	透明，后端无感知
postMessage	iframe 跨域通信	需双方代码配合
WebSocket	跨域双工通信	协议本身支持跨域

八、网络安全（前端视角）

8.1 XSS（跨站脚本攻击）

类型	说明	示例
反射型	恶意脚本在 URL 中，服务端未过滤直接返回	?q=<script>alert(1)</script>
存储型	恶意脚本持久化存储在服务器	评论区插入 <script>
DOM 型	前端 JS 直接解析 URL 中的恶意内容	location.hash 未转义

防御措施：

- 输入过滤 / 输出转义（HTML 实体编码）
- CSP（Content-Security-Policy）
- HttpOnly Cookie（防脚本读取）
- 避免 innerHTML、dangerouslySetInnerHTML

```
// ❌ 危险
element.innerHTML = userInput

// ✅ 安全（自动转义）
element.textContent = userInput

// ✅ React/Vue 默认转义
<div>{userInput}</div> // React 自动转义
```

8.2 CSRF（跨站请求伪造）

攻击流程：

1. 用户登录 A 银行
2. 用户未登出访问恶意网站 B
3. B 的页面自动发起请求：img src="bank.com/transfer?to=B&amount=1000"
4. 浏览器自动携带 A 银行的 Cookie，请求成功！

防御措施：

- SameSite Cookie: SameSite=Strict（最有效）
- CSRF Token：表单中嵌入随机 Token

- **Referer/Origin 验证**: 检查请求来源
- **验证码**: 敏感操作要求验证码

Set-Cookie: session=abc123; SameSite=Strict; Secure; HttpOnly

8.3 CSP (内容安全策略)

Content-Security-Policy:
default-src 'self';
script-src 'self' 'nonce-abc123' https://cdn.example.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data: https://*;
connect-src 'self' https://api.example.com;
frame-ancestors 'none';

指令	作用
default-src	默认策略
script-src	允许的 JS 来源
style-src	允许的 CSS 来源
img-src	允许的图片来源
connect-src	允许的 API 请求目标
frame-ancestors	允许嵌入的父页面 (防 Clickjacking)
report-uri	违规上报地址

8.4 HTTPS 中间人攻击

攻击方式: 代理服务器安装根证书，解密所有流量

防御:

- **HSTS** (HTTP Strict Transport Security) : 强制浏览器使用 HTTPS
- **证书锁定**: 仅信任特定证书
- **SRI**: 子资源完整性校验

HSTS (一次性告诉浏览器，未来半年强制 HTTPS)
Strict-Transport-Security: max-age=15768000; includeSubDomains; preload

九、Cookie / Session / Token

9.1 Cookie 属性

属性	说明
Name=Value	键值对
Expires / Max-Age	过期时间
Domain	域名限定
Path	路径限定
Secure	仅 HTTPS 传输
HttpOnly	JS 不可读取 (防 XSS)
SameSite	跨站请求策略: Strict / Lax / None

Cookie 大小限制: 每个域名最多 20-50 个, 每个最大 4KB

9.2 Cookie vs LocalStorage vs SessionStorage

特性	Cookie	localStorage	sessionStorage
容量	4KB	5-10MB	5-10MB
自动发送	✔ 同域请求自动携带	✗	✗
过期时间	手动设置	永不过期	关闭标签页清除
httponly	✔ 支持	✗	✗
作用域	域名 + 路径	域名	标签页

9.3 JWT 结构

```
JWT = Header.Payload.Signature
      ↓
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMONTY3ODkwIn0.doZjgSg4SA1sXzYq8s0E4P0GQ0A
```

安全实践:

- access_token: 短期 (15min) , 存储在内存 / sessionStorage
- refresh_token: 长期 (7天) , 存储在 HttpOnly Cookie
- Refresh Token Rotation (每次刷新更换 refresh_token)

十、HTTP 状态码

10.1 1xx 信息

状态码	含义	说明
100	Continue	继续发送请求体
101	Switching Protocols	协议切换（如 WebSocket 升级）

10.2 2xx 成功

状态码	含义	说明
200	OK	请求成功
201	Created	创建成功（POST）
204	No Content	无返回体（DELETE）
206	Partial Content	范围请求（视频分片）

10.3 3xx 重定向

状态码	含义	说明
301	Moved Permanently	永久重定向（搜索引擎更新 URL）
302	Found	临时重定向
304	Not Modified	协商缓存命中
307	Temporary Redirect	临时重定向（保持请求方法）
308	Permanent Redirect	永久重定向（保持请求方法）

10.4 4xx 客户端错误

状态码	含义	说明
400	Bad Request	请求格式错误
401	Unauthorized	未登录/无有效 Token
403	Forbidden	已认证但无权限
404	Not Found	资源不存在
405	Method Not Allowed	方法不允许
408	Request Timeout	请求超时

状态码	含义	说明
409	Conflict	资源冲突
429	Too Many Requests	请求频率限制

10.5 5xx 服务端错误

状态码	含义	说明
500	Internal Server Error	服务器内部错误
502	Bad Gateway	网关或代理错误
503	Service Unavailable	服务暂时不可用（过载/维护）
504	Gateway Timeout	网关超时

十一、WebSocket 与 SSE

11.1 WebSocket

webSocket 握手过程:

客户端 →
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13

服务端 →
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=

对比项	WebSocket	HTTP 轮询	SSE
通信方向	全双工	请求-应答	服务端→客户端
协议开销	低（轻量帧）	高（完整 HTTP 头）	低（HTTP 流）
实时性	最高	取决于轮询间隔	高
二进制	✓	✓	✗（仅文本）
自动重连	✗	✗	✓ EventSource

11.2 SSE (Server-Sent Events)

```
const eventSource = new EventSource('/api/events')

eventSource.onmessage = (event) => {
  console.log('收到数据:', event.data)
}

eventSource.addEventListener('custom', (event) => {
  console.log('自定义事件:', event.data)
})

eventSource.onerror = () => {
  console.log('连接断开 (自动重连中...)')
}
```

服务端格式:

```
Content-Type: text/event-stream

data: {"message": "hello"}

event: custom
data: {"id": 1}

retry: 3000
```

十二、HTTP/3 与 WebTransport

12.1 QUIC 协议特性

特性	优势
0-RTT 连接	首次 1-RTT, 后续 0-RTT 建立连接
用户空间实现	快速迭代, 不依赖操作系统更新
连接迁移	从 WiFi 切到 5G 不断连
前向纠错	减少重传, 提高弱网性能
无队头阻塞	一个流丢包不影响其他流

12.2 WebTransport (HTTP/3 上层协议)

WebTransport 是基于 QUIC 的全双工通信 API, 是 WebSocket 的下一代替代方案:

```
// WebTransport (实验性 API)
const transport = new WebTransport('https://example.com:443')

await transport.ready

// 单向流 (服务端→客户端)
const reader = transport.receiveStreams().getReader()
const { value: stream } = await reader.read()
const { value: data } = await stream.readable.getReader().read()

// 不可靠数据报 (类似 UDP)
const writer = transport.datagrams.writable.getWriter()
writer.write(new TextEncoder().encode('hello'))
```

对比	WebSocket	WebTransport
底层	TCP	QUIC (UDP)
可靠性	可靠流	可靠 + 不可靠
多路复用	一个连接一个流	多个流无队头阻塞
连接迁移	✗	✓
浏览器支持	全部	Chrome 正在推进

十三、面试高频问答

Q1：从输入 URL 到页面呈现的过程？

1. 用户输入 URL
2. 浏览器进程处理：检查是否合法 URL，否则搜索
3. DNS 解析（递归查询）
4. TCP 三次握手（HTTPS 还要 TLS 握手）
5. HTTP 请求（可能：缓存命中则直接返回）
6. 服务端处理并返回响应
7. 浏览器解析 HTML → 构建 DOM 树
8. 解析 CSS → 构建 CSSOM 树
9. 合并为 Render Tree
10. Layout（计算几何位置）
11. Paint（绘制像素）
12. Composite（合成图层）
13. ✓ 页面呈现

Q2: HTTP/2 的队头阻塞是什么? HTTP/3 怎么解决的?

HTTP/2 队头阻塞 → TCP 层面

└ TCP 保证有序传输, 一个包丢失 → 所有流都要等待重传

HTTP/3 (QUIC) 解决方案 → UDP 层面多路复用

└ 每个流独立传输, 一个流丢包不影响其他流

└ 应用层控制重传, 避免 TCP 层的串行阻塞

Q3: 为什么不用 Cookie 存 Token?

方案	安全性	灵活性	适用场景
<code>localStorage</code> 存 token	✗ XSS 可窃取	✓ JS 可控	不推荐
HttpOnly Cookie	✓ XSS 不可读	✗ JS 不可控	✓ 推荐
混合方案	✓ 两层防护	✓ 兼顾	✓ 最佳实践

Q4: HTTPS 中间人攻击怎么防?

防御层次:

1. 浏览器证书验证 → 拒绝自签名/过期证书

2. HSTS → 强制 HTTPS, 防 SSL Stripping

3. 证书锁定 → 只信任特定证书

4. SRI → HTML 引入资源时校验完整性

Q5: 状态码 301 vs 302 vs 307 vs 308?

状态码	含义	方法是否改变
301	永久重定向	POST → GET
302	临时重定向	POST → GET
307	临时重定向	方法不改变
308	永久重定向	方法不改变

Q6: CDN 缓存失效怎么办?

常见方案:

1. 版本号 (推荐): URL 携带版本参数或文件名 Hash

2. 强制刷新: CDN 控制台手动刷新节点缓存

3. 设置较短的 max-age + ETag 协商缓存

4. 带时间戳的 URL (不推荐, 破坏缓存)

Q7: WebSocket 心跳机制怎么设计?

```
// 客户端心跳
const ws = new WebSocket('wss://example.com')
let heartbeatTimer

function startHeartbeat() {
  heartbeatTimer = setInterval(() => {
    ws.send(JSON.stringify({ type: 'ping' }))
  }, 30000) // 30 秒一次
}

ws.addEventListener('open', startHeartbeat)

ws.addEventListener('message', (event) => {
  const data = JSON.parse(event.data)
  if (data.type === 'pong') {
    // 收到服务端 pong, 连接正常
  }
})

ws.addEventListener('close', () => {
  clearInterval(heartbeatTimer)
  // 自动重连...
})
```

关联文件

关联专题	文件
浏览器原理（安全/缓存详情）	01-浏览器原理.md
性能优化（网络层面）	02-性能优化.md
面试题库（网络专题）	01-前端面试题库.md
工程化（CI/CD/构建）	03-前端工程化.md

 **学习建议：**网络是前端面试的高频考点，建议结合抓包工具（Chrome DevTools Network 面板 + Wireshark）直观理解协议交互过程。

导航